ML-I LAB PROJECT

Banger or Not?

Predicting Hit Songs with Logistic Regression

Student	Sabalpara Krish
Roll number	D53
Enrollment number	24004501210368
Division	E
Project ID	P02
Model	Logistic Regression

1. Problem Statement

The aim of this project is to predict whether a song is labelled as a hit based on its audio features and genre. In this project, a song is labelled as a hit when its popularity score is 70 or higher. Songs with a score below 70 are labelled as non-hits.

Target rule: popularity >= 70 means hit = 1; otherwise hit = 0. The model predicts this label from the remaining features.

Dataset source: Kaggle - Spotify Tracks Dataset by maharshipandya.

2. Dataset Description

The original file contains 114,000 rows and 21 columns. It contains song information, audio measurements, popularity and genre. The dataset has 3 missing values and 0 exact duplicate rows before cleaning.

Target class counts

Non-hit (0): 108,528 songs (95.2%). Hit (1): 5,472 songs (4.8%). This is an imbalanced dataset because the hit class is much smaller.

Data dictionary and column decisions

Column	Meaning	Decision
Unnamed: 0	Row index from the CSV	Drop
track_id	Spotify track identifier	Drop
artists	Artist or artists name	Drop
album_name	Album name	Drop
track_name	Song title	Drop
popularity	Popularity score (0-100)	Use to create hit, then drop
duration_ms	Song duration in milliseconds	Keep
explicit	Whether explicit content is marked	Keep
danceability	Suitability for dancing	Keep
energy	Perceived energy level	Keep
key	Estimated musical key	Keep
loudness	Average loudness in dB	Keep
mode	Major or minor mode	Keep
speechiness	Amount of spoken words	Keep
acousticness	Acoustic character score	Keep
instrumentalness	Likelihood the track is instrumental	Keep
liveness	Likelihood of live performance cues	Keep
valence	Musical positivity score	Keep
tempo	Estimated tempo in beats per minute	Keep
time_signature	Estimated beats per bar	Keep
track_genre	Genre label	Keep; one-hot encode
hit (new)	Target created from popularity threshold	Target, not an input
duration_minutes (new)	Duration converted into minutes	Keep as derived feature

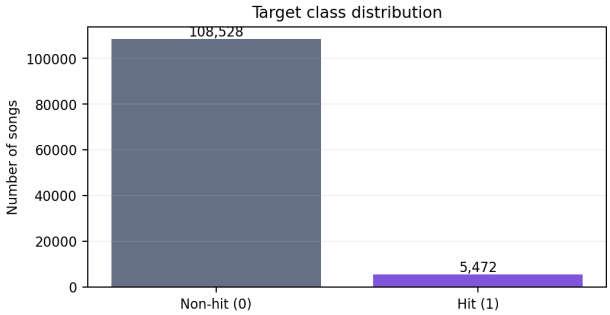
3. Data Cleaning and EDA

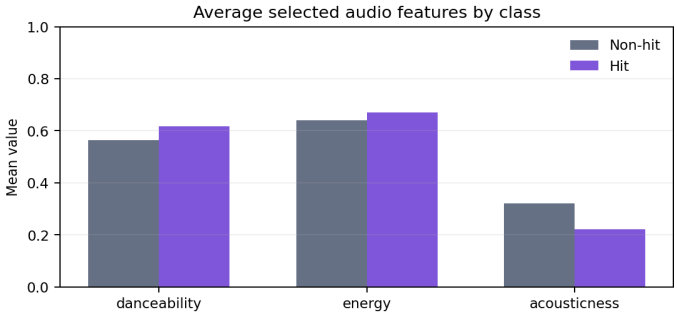
Data cleaning was done with Python and Pandas. No rows were removed as duplicates. The three missing entries were in artist, album and track name fields; those columns were dropped because they were not used as model inputs. The index and track ID columns were also dropped. Popularity was removed after creating the hit label to avoid giving the answer to the model.

Cleaning steps

- Created hit from popularity: 1 if popularity is 70 or above, otherwise 0.
- Removed duplicate rows and dropped index, identifiers and song-name columns.
- Dropped popularity after creating the target to prevent target leakage.
- Converted explicit to numeric form, one-hot encoded genre, and created duration_minutes from duration_ms.

Charts and five observations





- The dataset contains 108,528 non-hit songs and 5,472 hit songs. Only 4.8% are labelled as hits.
- Average danceability is 0.616 for hits and 0.564 for non-hits.
- Average energy is 0.671 for hits and 0.640 for non-hits.
- Average loudness is -6.66 dB for hits and -8.34 dB for non-hits. A less negative value is louder.
- Average acousticness is 0.221 for hits and 0.320 for non-hits.

These are patterns in this dataset. They do not prove that any one feature makes a song become a hit.

4. Model and Method

The project uses Logistic Regression, a classification model that estimates the probability that a song belongs to the hit class. It was selected because the faculty guide requires this model.

Feature preparation

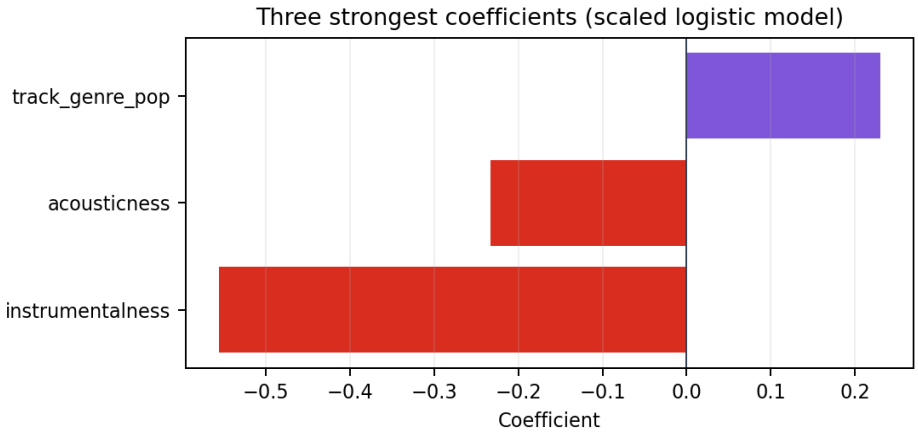
- Created a new feature, duration_minutes, by dividing duration_ms by 60,000.
- Separated the input features (X) from the target (y).
- One-hot encoded categorical genre values. After encoding, there were 128 input columns before feature selection.
- Split data into 80% training and 20% testing with stratify=y and random_state=42.
- SelectKBest with the ANOVA F-test selected 15 features using training data only.
- StandardScaler was fitted on training data and then used to transform both training and test data.

Settings compared

Setting	What was tested
Baseline	Default Logistic Regression
Scaled / Unscaled	Model with scaled and unscaled selected features
Threshold 0.3, 0.5, 0.7	Different probability cut-offs for predicting a hit
Weight=None / balanced	Regular training compared with class-weight balancing

Why recall and F1 matter

Only 4.8% of records are labelled as hits. A model can get high accuracy by predicting almost every song as a non-hit, while missing nearly all actual hits. Recall measures how many actual hits the model finds. F1 combines precision and recall into one score.



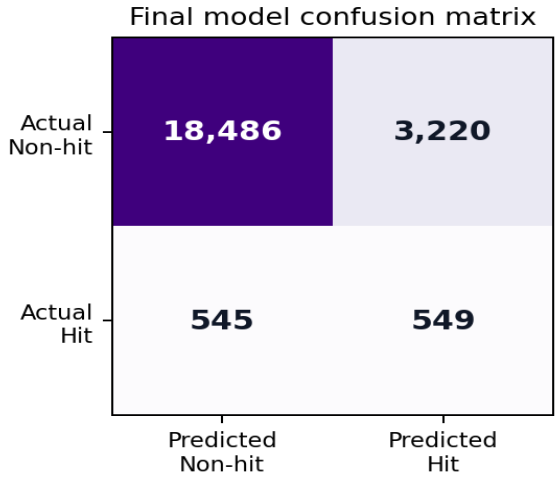
The coefficient chart shows the three largest absolute coefficients from the scaled Logistic Regression model. Coefficients show model associations, not proof of cause.

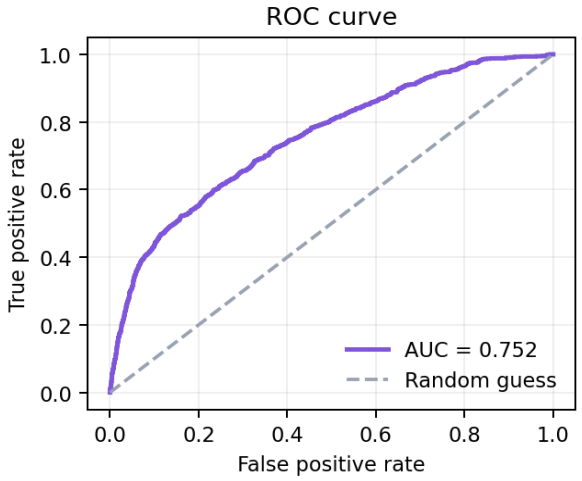
5. Results and Evaluation

The test set contains 22,800 songs: 21,706 non-hits and 1,094 hits. The settings were compared using the same stratified split.

Setting	Acc.	Prec.	Recall	F1	Spec.	AUC
Baseline	0.952	1.000	0.002	0.004	1.000	0.752
Scaled	0.952	1.000	0.002	0.004	1.000	0.752
Unscaled	0.952	0.000	0.000	0.000	1.000	0.752
Threshold=0.3	0.949	0.339	0.070	0.117	0.993	0.752
Threshold=0.5	0.952	1.000	0.002	0.004	1.000	0.752
Threshold=0.7	0.952	0.000	0.000	0.000	1.000	0.752
Weight=None	0.952	1.000	0.002	0.004	1.000	0.752
Weight=balanced	0.835	0.146	0.502	0.226	0.852	0.752

Selected setting: Weight=balanced. It was selected by sorting for hit-class Recall first and F1-score second. Its accuracy was 83.49%, precision 14.57%, recall 50.18%, F1-score 0.226, specificity 0.852, and AUC 0.752.





Confusion matrix: 18,486 non-hits were correctly identified, 3,220 non-hits were predicted as hits, 545 hits were missed, and 549 hits were correctly identified. The ROC-AUC score of 0.752 is above 0.5, but performance is not perfect.

6. Conclusion and Learning

The Logistic Regression model was trained to classify tracks as hit or non-hit using the target rule based on popularity. The selected setting was Weight=balanced. On the held-out test data, it achieved 83.5% accuracy and found 50.2% of the hit songs. It performed better at finding hits than the unweighted baseline, although many songs predicted as hits were actually non-hits.

What I learned

- How to inspect and clean a real dataset using Python and Pandas.
- How to create a target variable from an existing column.
- How one-hot encoding changes genre categories into numeric features.
- Why the dataset should be split before feature selection and scaling.
- How accuracy, precision, recall, F1-score, specificity and AUC describe model performance.
- How class imbalance affects predictions and why recall is important for a rare class.

Limitations

- The label is based on the dataset popularity score; it is not a guarantee that a song will become popular in the future.
- The data is imbalanced, so the model still misses some actual hits and produces false hit predictions.
- The result is measured on one 80/20 split of this dataset. Results may differ on new data.
- Audio features and genre alone may not capture promotion, artist audience, release timing or changing music trends.

Final note

This project shows the basic machine-learning process from loading and cleaning data to training, testing and evaluating a Logistic Regression model. The model gives a prediction, not a guarantee of a song becoming a hit.

Dataset: Spotify Tracks Dataset, Kaggle. Target definition and model workflow follow the faculty-provided P02 project guide. Metrics in this report were reproduced from the supplied dataset using the described split and feature-preparation workflow.